

AI-Assisted Design of a Post-Quantum Cryptographic Accelerator: A Deployed-Silicon Case Study

Jungmin Park, Eunha Kim, Wooseop Kim, Seongjoon Cho, and Byungho Cha

Abstract—Post-quantum migration is mandated on published timelines, and silicon that ships with a defect cannot be patched remotely. The standard acceptance gate cannot detect an entire class of ML-DSA defects. Signing resamples until a candidate meets its norm bounds, so the executed path varies with the message, whereas known-answer tests (KATs) sample fixed values and reach only the depths their seeds trigger. Our accelerator passed its full KAT regression while carrying a norm check that outran block-RAM latency, leaving each candidate’s final coefficients unverified; the escape surfaced at reject-loop iteration 5. The blind spot lies in the instrument, not the engineer; care cannot remove it. We replace that gate. A byte-exact golden-reference oracle paired with randomized adversarial soak drives the rejection loop past any fixed vector, closing the gap: 301,343 data-dependent signings, zero escapes. Because the gate judges artifacts and never authors, trust becomes separable from authorship, making AI authorship an answerable question. We report 232 logged experiments in which an agentic large language model drove a unified ML-KEM-768 and ML-DSA-65 accelerator with on-chip key custody from RTL to PCIe bring-up on one Kintex-7 XC7K160T, shipped at 98.5% slice occupancy. Success was 71.6%, following a hardware-coupling gradient, 77–85% for documentation and research against 50–53% for synthesis and bring-up, which observability can explain: failure concentrates where corrective signals are physical-side only. That so unreliable an author produced an artifact byte-exact across all six FIPS operations—its deployed baseline surviving the same 779,945-check zero-failure soak—is the claim.

Index Terms—Post-quantum cryptography, FPGA, LLM-assisted design, design methodology, empirical study, hardware security module, ML-KEM, ML-DSA, NTT, key custody, agentic AI, design automation

I. INTRODUCTION

Post-quantum cryptographic hardware is essential yet error-prone, and the test that decides whether it may ship is blind to an entire class of its defects. NIST closed PQC standardization in August 2024 with FIPS 203 (ML-KEM [1], [2]) and FIPS 204 (ML-DSA [3], [4]). CNSA 2.0 [5] and NIST IR 8547 [6] attach dates, the latter proposing deprecation of 112-bit-security RSA and ECC by 2030 and disallowance of all quantum-vulnerable algorithms by 2035. Migration is therefore a scheduled compliance obligation, and it lands hardest on hardware. Roots of trust, secure elements, and signing devices stay in the field for a decade or more and

cannot be patched remotely. A defect shipped in silicon is a recall, not a release.

The defect class that concerns us is structural: ML-DSA signing is rejection-sampled. The signer resamples until the candidate satisfies its norm bounds, so the number of iterations, and with it the control path exercised, depends on the message. Correctness is therefore a property of execution *paths*. Known-answer tests sample *values* instead, and a fixed vector set reaches only the reject-loop depths its own seeds happen to trigger. Enlarging that set does not help: the coverage limit follows from how the vectors are built, not from how many there are.

Our accelerator passed its full KAT regression while carrying BUG-IDE-037: the rejection-sampling norm check iterated over all 256 coefficients of each polynomial, but block-RAM read latency delivered the final one or two *after* the accept/reject decision (Section V-D). The defect was reachable only at reject-loop iteration 5,¹ for one message class, and every affected signature failed verification. It is one of eight defect classes we catalogue, each caught by a validation layer the others miss (Table II).

The gap lives in the instrument: a careful team using KAT as its acceptance gate ships the same defect. We therefore replace the gate with a byte-exact golden-reference oracle paired with randomized adversarial soak that drives the rejection loop past any fixed vector’s reach. Because the replacement judges artifacts and never authors, it is *author-agnostic*: who wrote the RTL becomes a separate question.

That separation is what makes the AI question worth asking. The deadline is fixed, engineers fluent in both lattice cryptography and FPGA timing closure are scarce, and agentic LLM coding assistants are already entering hardware flows. The open question is not whether they will be used on PQC silicon but *where in the flow they can be trusted*. The LLM-for-RTL literature evaluates snippets and single modules by pass@*k* at simulation or lint [7], [8], [9], and even agentic, tool-in-the-loop flows [10], [11], [12], [13] stop short of validated, hardened operation on deployed silicon. No published study reports how assistant reliability varies *across the phases* of a real campaign.

Across the 2026 development cycle (2026-01 through 2026-06) an agentic LLM assistant drove design, verification,

J. Park and B. Cha are with Lucid Motors, Newark, CA, USA. E-mail: jungminpark@lucidmotors.com; byunghocha@lucidmotors.com.

E. Kim, W. Kim, and S. Cho are with EYL Inc., Seoul, Republic of Korea. E-mail: ehkim@eylpartners.com; wskim@eylpartners.com; jhcho@eylpartners.com.

¹BUG-IDE-NNV labels are the project’s stable defect-log identifiers. Iterations count signing resampling rounds: the fifth candidate was the first whose only out-of-bound coefficient fell in the unchecked tail, so the device accepted what a correct signer rejects (first correct accept: iteration 8).

synthesis, host-software development, silicon bring-up, and security hardening of a unified ML-KEM-768 + ML-DSA-65 accelerator with on-chip key custody, deployed on an XC7K160T Kintex-7 over PCIe Gen2 $\times 8$. All 232 experiments were logged. “Silicon” herein means the physically deployed FPGA in live operation, not simulation.

Three constraints made that campaign adversarial. *Area* was binding. Both NIST Level-3 schemes plus the custody HSM had to fit one mid-range XC7K160T, which forced a serialized number-theoretic transform (NTT) and still left the shipped build at 98.5% slice occupancy. The limit was placement density rather than arithmetic (Sections V-B and IV-F). *Timing* was tight. The 500 MHz domain of the Xilinx XDMA PCIe bridge sets the design’s worst-case setup slack at +0.049 ns (0 failing among 178,247 endpoints design-wide); the path tipped negative during bring-up and was recovered by a seed sweep because the cause was congestion, not logic. *Observability* was the worst of the three. A stale command register survived soft-reset, wedging DMA ~ 1 -in-20 yet staying hidden for weeks because the soak harness self-healed it. That did not stop the assistant from reaching a confident, plausible, and incorrect diagnosis in ten logged episodes (Section IV-F).

Where the corrective signal for a task sits in an artifact the assistant can read, it performs at software-domain rates. Where that signal exists only on the physical side of the boundary, in DMA-engine residue, reset-domain asymmetry, placement density, or silicon-versus-simulator semantics, no reasoning over readable artifacts recovers it, and reliability falls to roughly a coin flip. This *hardware-coupling gradient* is a claim about *observability*, not that hardware is harder; its remedy is to make the physical side readable.

This work makes three contributions:

- (C1) **A validation construction that closes a gap KAT cannot reach.** Fixed-vector testing is structurally incomplete for rejection-sampled signature hardware; we demonstrate the consequence on deployed silicon (BUG-IDE-037, passed by a full KAT regression) and close it with the oracle-plus-soak gate above, over 301,343 data-dependent signings with zero escapes. The construction applies to any ML-DSA hardware effort; we generalize it into nine honesty constraints under which trust derives from validation artifacts, never from authorship (Section VIII).
- (C2) **The hardware-coupling gradient and its cause.** From 232 structured logs (40 task categories, 76 milestones) we show that LLM success declines as tasks approach FPGA placement, timing, and live-hardware state (77–85% documentation and research; 50–53% synthesis and bring-up), and we identify the mechanism: failure concentrates where corrective information is unobservable in any artifact the model can read. We also report an eight-class bug taxonomy and ten “confidently wrong, then corrected” episodes (Sections IV-E and IV-F).
- (C3) **Silicon existence proof under binding constraints.** The loop produced a unified ML-KEM-768 + ML-DSA-65 accelerator plus key-custody HSM co-resident on one XC7K160T at 98.5% slice occupancy with 49 ps of setup margin at 500 MHz and hardware-RNG-seeded keys under a device-bound key-encryption key (KEK)—

a combination that is, to our knowledge, unique among unified PQC accelerators. The artifact is byte-exact across all six FIPS operations, survived a zero-failure 779,945-check soak on the deployed firmware baseline v88 (vN labels successive firmware builds), and was productized through two releases to the final FIPS-203/204 wire formats (Sections V–VI). That an author measured at 71.6% task success produced an artifact with zero validation escapes is the concrete demonstration of (C1).

Efficiency here means consolidation rather than speed: both schemes plus the key-custody hardware security module (HSM) share one runtime-modulus-switched NTT/INTT datapath and one Keccak-f[1600] core. *Security* means key custody: private keys are generated, wrapped, and used on-die, device-bound and adversarially validated. The claim is device binding, not at-rest secrecy and not side-channel resistance. Table V places the artifact against the closest unified ML-KEM+ML-DSA accelerators on exactly that basis.

Two 2026 works combine LLMs with PQC hardware, LLM4PQC [14] and Liao et al. [15], the latter the first quantitative LLM-versus-HLS comparison (up to $2.6\times$ kernel speedup); the delta is one of depth rather than kind, contrasted axis by axis in Table VI (Section VII-D).

The paper proceeds from background (Section II) through the methodology (Section III), the empirical record (Section IV), and the artifact with its PCIe system (Sections V–VI) to related work (Section VII), threats to validity (Section VIII), and the conclusion (Section IX).

II. BACKGROUND

A. Preliminaries

ML-KEM-768 (FIPS 203) is a Module-LWE (MLWE) key-encapsulation mechanism with rank $k = 3$ over $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$, $n = 256$, $q = 3329$, $\zeta = 17$. Hashing uses SHAKE128/256 and SHA3-256/512 (FIPS 202 [16]). Wire sizes are encapsulation key 1184 B, decapsulation key 2400 B, ciphertext 1088 B, and shared secret 32 B.

ML-DSA-65 (FIPS 204) is a Fiat–Shamir-with-aborts lattice signature (MLWE and Module-SIS problems) with $n = 256$, $q = 8380417$, $\zeta = 1753$, $(k, \ell) = (6, 5)$, SHAKE256 mask expansion, and rejection-sampling bounds $\gamma_1 = 2^{19}$, $\gamma_2 = (q-1)/32$, $\beta = 196$, $\omega = 55$. Wire sizes are public key 1952 B, secret key 4032 B, and signature 3309 B.

Shared-primitive opportunity. Both schemes share $\mathbb{Z}_q[x]/(x^n + 1)$, $n = 256$: the NTT butterfly is identical in form across both moduli, and both rely on the same Keccak permutation, so resource sharing is natural and both schemes fit together on a Kintex-7 XC7K160T (Section V-B).

B. LLM-Assisted Hardware Design: A Brief Primer

LLMs can generate syntactically valid Verilog [7], [8], [9], close simulation-feedback loops [10], [11], [12], [17], and assist verification and security-sensitive RTL repair [18], [19] (surveyed in Section VII). To our knowledge, no prior work closes the full path (RTL generation, synthesis, host software, live-hardware bring-up, iterative hardening) on a complete cryptographic system while reporting a campaign-level longitudinal dataset of LLM-task outcomes.

III. METHODOLOGY: AN AGENTIC LLM-ASSISTED DESIGN LOOP

Figure 1 gives the whole picture. Candidates passing the human review gate climb the byte-exact golden-reference validation spine of Section III-D.

A. The Assistant and Its Toolchain

The design assistant is Claude Code, an agentic command-line interface giving the LLM file-system and shell tools and the ability to spawn specialist subagents. On wide-scope integration days an Opus-4.6 team-lead orchestrated multiple Sonnet-4.6 subagents. All models were Claude-family: Opus-4.8 (93 logs, 17 with a 1M-context window), Opus-4.7 (63), Opus-4.6 (47), Opus-4.5 (24), Sonnet-4.6 (3, specialist subagents), and Fable-5 (1, deep research); counts are per-log model attributions (one log lacks a tier).

The assistant wrote Verilog-2001 RTL in `rtl/`. It ran Icarus Verilog (iverilog) simulation with GTKWave review on failures, and Vivado 2024.2 batch synthesis and implementation via assistant-generated Tcl (`ExtraNetDelay_high` placement for congested respins, per-clock Intra-Clock-Table timing analysis). It maintained Python golden references for FIPS 203 and FIPS 204 (round-3 wire formats through v88, FIPS-final thereafter; Section VI-E), byte-compared against simulation and silicon. It drove Python (`pqc_pcie_host.py`) and C99 (`sw/c/`) host libraries over XDMA PCIe DMA. A separate agent thread ran multi-agent adversarial review of critical design premises, catching at least one wrong optimization premise (EXP-20260604-001, Section IV-F).

B. Context Architecture: The Persistent Markdown Stack

Prompts were typically one to three sentences plus pasted tool or hardware output; the harness injects a four-layer stack of persistent Markdown files into every session—three layers versioned in the repository, plus a typed memory in the harness’s per-project store.

- **Project constitution (CLAUDE.md, ≈480 lines; P2).** Repository layout, Verilog conventions, the XC7K160T resource budget, canonical tool commands, the logging template of Section III-C, and both schemes’ mathematical parameters.
- **Typed persistent memory (42 topic files at the 2026-06-26 corpus snapshot, plus a one-line-per-entry index; P5).** Each file records a single fact in YAML front matter, written unprompted by the assistant. The 21 *project* files hold live status; the 21 *feedback* files hold failure-distilled engineering rules with explicit *why* and *how-to-apply* fields—e.g., “on a ≈86%-LUT, sub-0.1-ns-WNS design, even a no-op scaffolding port can break unrelated timing,” which later prevented repeat failures.
- **Specialist-agent definitions (seven role files).** Role prompts with restricted tool allowlists. The fan-outs of Section III-A instantiate these.
- **Procedural skills (five skill files).** Checklists invoked as named commands—e.g., `/update-all` and `/spawn-team`—making the logging discipline of Section III-C mechanical.

In our experience the *feedback* layer carries the most methodological value: a lesson learned in week n changes assistant behavior in week $n+k$ (Section IV-F, lesson themes). Because the layers outlive any conversation, work resumes after total context loss without re-teaching.

One further persistent artifact sat outside the stack: a plan-and-milestone roadmap in the project dashboard’s timeline data, decomposing the six-month effort into an ordered sequence (M1–M8bp, 76 entries; Fig. 2). Each milestone’s “done” is verifiable, so a resuming session inherits the last closed increment and the next target. M1–M8 were fixed through the v1.0 release; M8a–M8bp were appended afterward for the unanticipated silicon bring-up, key-custody, and productization work. We therefore read its role as purely *structural* and do *not* claim the plan raised the per-task success rate ($n=1$; Section IV-G).

In Figure 2, M1–M8 span pre-silicon to first release (weeks 3–12): architecture locked (M1), NTT verified (M2), all modules done (M3), Kyber (M4) and Dilithium (M5) integrated (campaign-era scheme names), optimization (M6) and validation (M7) complete, and the accelerator-only repository release (M8; distinct from the productized Release v1.0 of Section VI-E). M8a–M8bp group into five themes: (i) *PCIe hardware bring-up*: a Gen2×8 link and live XDMA (M8a; first bitstream 2026-05-04), then all six PQC operations byte-exact at full back-to-back PCIe throughput. (ii) *On-chip entropy*: an HRNG audit, QRNG/hybrid modes, and an internal-seed Key-Gen whose seed never crosses PCIe. (iii) *Cycle reduction*: the 125 MHz v69 respin and the v70–v77 datapath work. (iv) *On-die key custody* (v79–v87): sign-from-vault, on-chip key-wrap/export, a KEK rooted in the FPGA’s factory-programmed Device-DNA identifier, and SHAKE-KDF, capped by a TLS-1.3 CertificateVerify through hardware custody. (v) *Productization and hardening*: the BUG-IDE-037 fix in v84, exhaustive and eight-hour soaks, a native-C host library, the v88 Am-241 quantum-RNG baseline, and the v1.0 product release. The campaign closed with the FIPS-203/204 finalization—firmware v89 and the v1.1 release (ML-KEM-768/ML-DSA-65)—Fig. 2’s second deployment star.

C. Experiment-Logging Discipline

Every significant unit of work was recorded as structured JSON in `llm_logs/experiments/`, following the nine-field schema of Section III-G; per-field presence accompanies each count in Section IV. Logging was enforced at experiment close, not retrospectively.

Data availability. The experiment logs (JSON), milestone index, 40-category-to-ten-bucket mapping, and the scripts producing the tables of Section IV are available from the authors, credential-redacted; the logging schema, the methodology template of Section III-G, and the complete v89 hardware-validation transcript (Section VI-E) are supplementary material.

D. The Golden-Reference Validation Spine

No RTL reached a hardware build without byte-exact agreement with the Python golden reference in simulation. The

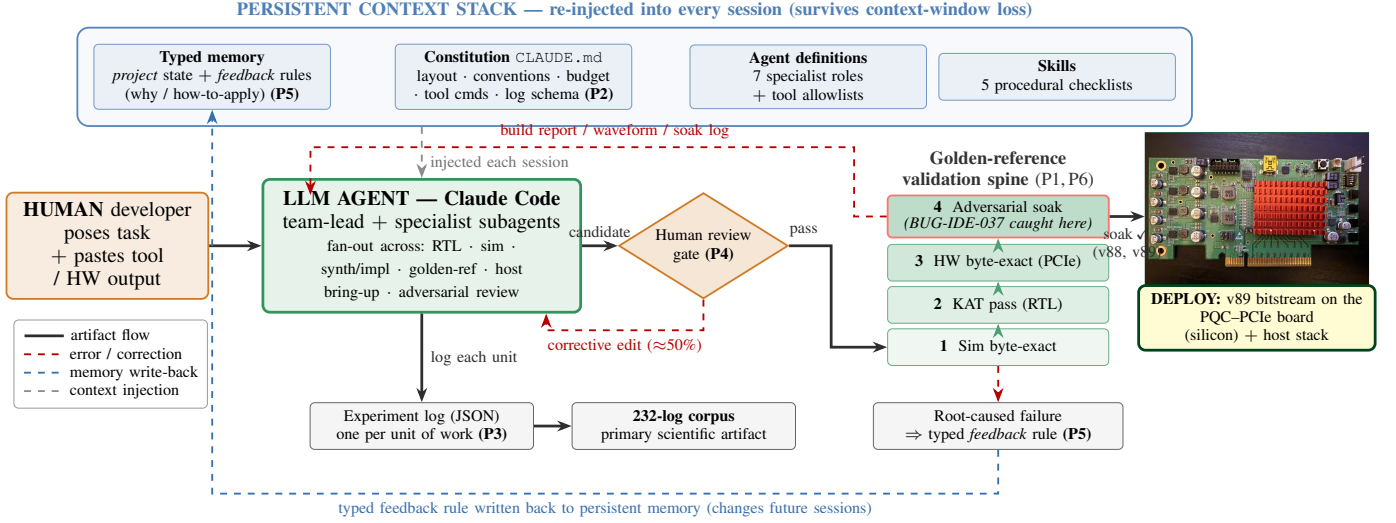


Fig. 1. The LLM-assisted hardware-design methodology as a closed *generate-validate-learn* loop ending in deployed silicon. Tags **P1–P6** mark the reusable-protocol steps of Section III-F. The rung-4 pass feeding deployment is the v88 baseline’s 779,945-check soak; the shipped v89 re-passed the full ladder, adversarial soak included (Section VIII, H3).

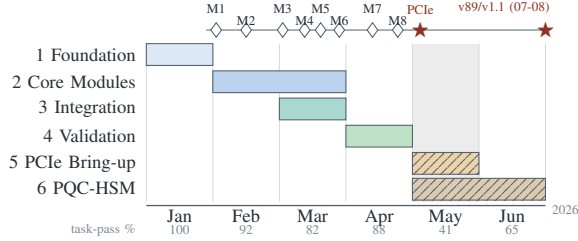


Fig. 2. Campaign timeline of the deployed Kintex-7 160T work (2026-01-06): six phase bars, planned milestones M1–M8 (diamonds), and two deployment landmarks (stars; the v89/v1.1 release of 2026-07-08 is drawn at the axis edge), all detailed in §III-B. The faint monthly task-pass % (§IV-G) reflects task mix, not a learning trend ($n=1$).

promotion ladder was: Sim byte-exact $\rightarrow$ KAT pass (RTL) $\rightarrow$ HW byte-exact (PCIe) $\rightarrow$ adversarial soak. The adversarial soak (Section V-D, reliability closure) exercises randomized message classes outside the KAT vectors; it caught BUG-IDE-037, invisible to all earlier rungs.

E. Human-in-the-Loop Role

The developer (the first author) posed tasks, *gated* every generated artifact (no output entered the simulator, synthesis, or the board without review; $\approx 50\%$ of edit-tracked generations needed a corrective edit, Section IV-D), supplied build reports and waveforms as corrective feedback, decided when to roll back, and made all *real hardware decisions*. The claim under test is that the agentic loop *with these human gates*—not unaided model authorship—carried the complete flow to deployment.

F. A Reusable Protocol for Other Implementations

Applying the loop to another scheme, accelerator domain, or FPGA/ASIC flow takes six steps:

P1. Executable golden reference first. Before any RTL, byte-exact agreement with the reference (here, Python FIPS 203/204) is the sole acceptance criterion for every artifact (Section III-D).

P2. Write the constitution. One versioned Markdown file (Section III-B) of layout, conventions, budget, tool commands, and logging schema.

P3. Instrument before generating. Adopt the experiment log (Section III-C) from day one.

P4. Gate by hardware coupling. Every artifact crosses the review gate (Fig. 1); concentrate review *depth* where physical coupling is high (the 50–53% buckets of Table I), lighter-touch where low (77–85%). Here every artifact was gated (Section III-E); the coupling–success ordering is offered as a default prior, not a validated predictor.

P5. Convert every root-caused failure into a typed feedback rule with why/how-to-apply fields (Section III-B).

P6. Promote only up the ladder of Section III-D; never skip a rung.

The scheme-specific investment was the golden reference and KAT vectors. The device-specific investment was the constitution’s tool-command and resource-budget sections. Everything else (the memory-stack *schema*, agent roles, logging schema, and promotion ladder) remained unchanged through the corpus-frozen v88 and the shipped v89. Whether it transfers with a similar success profile is untested (Section VIII, H1–H2).

G. The Methodology as a Reusable Template

The concrete artifact of the protocol in Section III-F is a repository-skeleton template holding the constitution `CLAUDE.md` (P2), the golden reference `ref/` (P1), the typed-memory files (P5), the agent-role and skill *scaffolds*, and the JSON experiment-log schema (P3). Only `CLAUDE.md` and `ref/` are rewritten per target; the remaining layers copy unchanged, and the memory contents and concrete role/skill files accrue as the project earns them (the template ships one example of each).

TABLE I
SUCCESS RATE BY TASK CATEGORY (232 EXPERIMENTS). *Model* IS THE MOST-USED CLAUDE TIER(S) LOGGED PER CATEGORY.

Category	<i>N</i>	Success	Rate	Model
Documentation	53	45	85%	4.5–4.8
Host-software / tooling	10	8	80%	4.7/4.8
Release / infra / PM	14	11	79%	4.6/4.8
Research / architecture	13	10	77%	4.5/4.6
Bug-fix / debug	47	35	74%	4.7
RTL generation	7	5	71%	4.7
Verification	33	23	70%	4.8
Review / audit	3	2	67%	4.8
Synthesis / optimization	38	20	53%	4.8
Integration / bring-up	14	7	50%	4.8
Total	232	166	71.6%	—

IV. EMPIRICAL RESULTS

A. Corpus and Overall Success Rate

The 232-experiment corpus, frozen at the 2026-06-26 analysis snapshot, records one intensive campaign (2026-01 to 2026-06) spanning 40 task categories and 76 named milestones (M1–M8bp).² Of the 232, 166 score success and 66 failure, an overall success rate of **71.6%** (Wilson 95% interval 65–77%). Generated or modified code sums to $\approx 26,373$ lines.

`syntax_correct True` appears in 166 logs, `simulation_pass True` in 107, `synthesis_pass True` in 106, and `functional_correctness passed` in 118 of the 127 logs carrying the field. Text explicitly claiming “HW-validated,” “byte-exact,” or “silicon” appears in 93 logs.

An experiment scores *success* when its log records the task’s acceptance criterion met at close, *after* any in-experiment iteration and human edits: functional correctness where a functional oracle exists (byte-exact against the FIPS reference, a passing simulation, or on-silicon validation), the applicable pass field otherwise. Abandoned or rolled-back outcomes score failure. The 49.2% manual-edit rate of Section IV-D measures intervention *en route*. The `syntax_correct` field is not an independent gate: it is recorded (always True) exactly in the 166 accepted logs and omitted in the 66 failures, so its count coinciding with the success count is definitional.

B. Success by Task Category

Table I presents the ten task buckets.

Finding. *LLM success rate falls the more directly a task touches FPGA placement, timing, and live-hardware state.* We ranked the ten buckets by coupling to live silicon—documentation (rank 1, no tool execution) through research/architecture, release/infra, host-software, review/audit, verification, RTL generation, bug-fix/debug, and synthesis/optimization to integration/bring-up (rank 10, live-silicon state required)—a ranking fixed from the coupling criterion alone (so not circular), giving Spearman $\rho \approx -0.83$ against the measured rates. This is a *descriptive* pattern, not a controlled

²Eight planned milestones (M1–M8, through the v1.0 release) plus 68 finer-grained post-plan sub-milestones (M8a–M8bp, in spreadsheet-column labeling) tracking the silicon bring-up, key-custody, and productization work.

measurement: the ranking is subjective, the buckets are neither independent samples nor equal-sized, and the corpus is $n=1$, so we attach no significance test. Model choice is an unlikely explanation, since the two lowest-success buckets were the most Opus-4.8-concentrated—our strongest tier (Table I, *Model* column). Even so, at $n=1$ coupling and model quality cannot be fully separated (Section VIII, H2).

The ten buckets coarse-grain the 40 logged `task.category` labels by keyword (segmentation and *oracle-stringency* caveats: Section VIII, H1). Per-bucket Wilson 95% intervals are wide at these sample sizes (documentation 73–92%, synthesis/optimization 37–68%); only the documentation–synthesis/optimization intervals are disjoint.

C. Iterations and One-Shot Rate

Iteration counts were recorded in 179 of 232 logs.³ The one-shot rate is 57.0% (102/179); 74.3% closed within two iterations and 93.3% within five, with mean 2.12, median 1, and maximum 14. A 95% interval on 102/179 spans roughly 50–64% (53-log missingness caveat: Section VIII, H1). A higher count reflects rework, not failure (Section IV-A).

The three hardest tasks by iteration count were EXP-20260324-001 (14 iterations, Dilithium signature top-level integration)⁴, EXP-20260522-001 (9, PCIe Dilithium Sign+Verify debug), and EXP-20260318-002 (8, Kyber KEM integration, six bugs in `kyber_decaps.v`).

D. Human-in-the-Loop Rate and Footprint

Correction rate. Manual-edit records appear in the *same* 179 logs that record iteration counts (Section IV-C). Of the 179, 91 (50.8%) report “None” (accepted as produced) and 88 (49.2%) at least one human correction. Intervention types (among the 87 with fine-grained tags; multi-tag, sums exceed 87): logic/RTL race correction 24, documentation fix 19, host-SW fix 18, syntax/compile fix 17, parameter tuning 15, timing/placement adjustment 12, rollback/revert 8, integration wiring 6.

Interpretation. The largest classes—RTL logic/race correction, host-SW fixes, timing/placement adjustments—are those where a software-trained model is structurally weakest, a weighting directionally consistent with the bucket-level finding (Table I); corrections nonetheless span all task types (raw counts; base-rate/task-mix caveat: H2).

The human-in-the-loop footprint. The 49.2% edit rate quantifies only one of five operator roles, which the logs measure unevenly:

- *Corrective edits* (measurable): the 49.2% edit rate above (worst-case missing-not-at-random bounds 37.9–60.8% over the full 232, from the 53 unrecorded logs) and the 2.12 mean iterations of Section IV-C.

³An iteration is one generate–review–correct cycle, from a log’s optional count field (`iterations_needed`, or `total_iterations` in early-schema logs; 1 = a one-shot acceptance). 53 of the 232 experiments in Table I leave it unrecorded, across many task categories rather than one type.

⁴Log-derived task names retain the campaign-era CRYSTALS names (Kyber, Dilithium); the text otherwise uses ML-KEM/ML-DSA for the standardized schemes.

- *Rollback / restart decisions* (partly measurable): 8 logged reverts.
- *Go/no-go hardware acts* (weakly measurable): every flashed bitstream and accepted timing closure is an irreversible operator decision—dozens across the bitstream lineage through v88—but not individually logged.
- *Task specification and artifact review/gating* (not measurable as a fraction): the operator wrote every prompt, gated every artifact entering the toolchain (Section III-E), and committed 100% of changes—uniform across *all* experiments, and plausibly the largest human inputs.

Self-recording and authorship-apportionment caveats are stated as threats (Section VIII, H1–H2).

E. Bug Taxonomy

Table II enumerates the project’s eight defect classes. Each validation layer catches a class the others miss, and the blindness is instrument-specific: simulation misses what only silicon shows, KAT misses what only randomization shows, and synthesis silently mis-elaborates RTL that simulates clean. Three of these families dominate the bring-up record and are analyzed in Section VI-C.

F. Ten Episodes of “Confidently Wrong, Then Corrected”

In ten episodes the LLM (or the developer acting on its output) reached a confident, incorrect conclusion, acted on it, and later corrected course—the hazard of plausible but wrong diagnosis. Five in detail:

(1) **Premature “zero-failures” claim** (EXP-20260614-001; corrected in EXP-20260615-001): a $\approx 84,000$ -check soak prompted it; later soaks surfaced BUG-IDE-037, misdiagnosed twice until proven deterministic (Section V-D). (2) **A “driver flake” that was hardware** (EXP-20260619-001): BUG-IDE-021, managed for months as such, was a stale HSM_CMD level register surviving a soft-reset. (3) **Draining the wrong FIFO** (EXP-20260620-002): a stuck C2H (card-to-host) path drew two wrong “drain the FIFO” remedies; ≈ 1041 B sat in the C2H *engine*, not the char FIFO—fixed by a burn-op (`_warmup_c2h`). (4) **“LUT-bound” that was placement density** (EXP-20260604-001): a multi-agent adversarial review overturned the “LUT-bound / idle DSPs” premise; the actual constraint was 97.49% slice occupancy. (5) **An “occasional” bug that was deterministic** (EXP-20260529-001): Dilithium Verify failures called “occasional contamination” were 200/200 deterministically wrong (BUG-IDE-036, Table II). The other five: EXP-20260625-001 (“other board broken”—it was the best); EXP-20260619-004 (a `ram_style="block"` fix staged, then infeasible on fallback); EXP-20260617-002 (a wrong on-die `sk` blamed on RTL; two sims showed host truncation); EXP-20260613-002 (a KDF “would not fit” the K160T; empirically wrong); EXP-20260422-006 (a correct BRAM fix reverted, then unreverted).

Lesson themes. Manual thematic coding (not a mechanical keyword count) of `lessons_learned` across the 232 logs yields eight themes (approximate log tallies in parentheses; multi-theme, so tallies sum past 232), each stated as an

applicable rule: (i) *verify the specific buggy phrase or count, not the bare token* (75)—generic searches always “pass”; grep the exact wrong string and check its count; (ii) *registered-vs-combinational / NBA discipline* (73)—a handshake output derived from registered state must itself be registered; iverilog tolerates what silicon will not (Section IV-E); (iii) *timing-as-congestion* (71)—near the area ceiling, negative slack is placement luck: sweep Explore seeds from one checkpoint, not `phys_opt`; read the per-clock timing tables; (iv) *host-vs-RTL boundary discipline* (37)—a boundary symptom does not name the layer that produced it; A/B the host-side variable before any RTL hunt (Section VI-C); (v) *read the build report, not historical docs* (35)—area and clock numbers go stale in prose, never in the report; (vi) *honest scoping* (29)—state the validation rung that earns a claim and claim nothing above it; (vii) *adversarial review to challenge premises* (28)—the costliest errors were confident premises, so red-team the diagnosis, not just the patch; and (viii) *a golden-reference sim before hardware* (22)—a byte-exact, X-clean model of every change before any bitstream.

G. Success Rate Over Time

Monthly success (Fig. 2) ran 100% (22/22), 92% (12/13), 82% (14/17), and 88% (45/51) over 2026-01 through 2026-04. It then dipped sharply to 41% (18/44) in 2026-05, the hardest phase, which combined PCIe Dilithium Sign+Verify bring-up, the HRNG audit, and a cycle-optimization campaign with several sub-floor or rolled-back builds. It recovered to 65% (55/85) in 2026-06 with the shift back to hardening and documentation.

Interpretation. The trajectory is not a learning curve. Live-hardware tasks were impossible before the first PCIe bitstream (2026-05-04, Section VI-C), so the monthly trajectory is largely the coupling gradient of Table I expressed in time, not an independent confirmation of it. Concurrent model-version changes (Section III-A) and phase effects such as accumulating design debt are further confounds $n=1$ cannot separate, though the bucket-level pattern favors the task-mix reading. This is the empirical form of contribution (C2).

H. Compute Cost: Token Consumption

Measurement window. The harness records per-message token usage, but transcripts of inactive sessions are pruned after roughly a month. What survived was one continuously active session spanning the final six weeks (2026-05-17 to 2026-07-02): 117 of the 238 experiments logged by the measurement date (six beyond the frozen 232-experiment snapshot used elsewhere), i.e. 49% of all logged campaign activity, covering the PCIe custody, soak-hardening, and documentation phases.

Token totals. Deduplicating usage records by message ID across the main session and all 615 subagent and workflow threads gives 17,877 assistant turns (9,303 main-session, 8,574 subagent) processing ≈ 4.47 billion tokens: 6.3 M fresh (uncached) input, 21.6 M generated output, 145.1 M prompt-cache writes, and 4,292.2 M prompt-cache reads.

TABLE II
BUG TAXONOMY OF DEFECT CLASSES ENCOUNTERED ACROSS THE CAMPAIGN, WITH DETECTION METHOD.

Class	Representative instances	Detection method
Non-blocking-assignment (NBA) / registered-vs-combinational handshake race	Multi-write BRAM (Vivado drops one write; BUG-IDE-026); shift-by-1 corruption from combinational <code>reg_out</code>	Simulation (iverilog); Vivado silently mis-synthesizes iverilog-clean RTL
Host-vs-RTL misdiagnosis	BUG-IDE-031/033/034: apparent RTL regressions were host-side (Section VI-C). BUG-IDE-036: retained Verify-FSM state across back-to-back calls—RTL-behavioral (reproduced in pure RTL simulation, 2026-08), mitigated by the per-call soft-reset (D1)	Live hardware A/B testing
Timing-closure / placement congestion	XDMA <code>userclk1</code> 500MHz path tipped negative; fixed by Explore seed-sweep, not <code>phys_opt</code>	Build timing report (per-clock Intra-Clock-Table)
FIPS / canonical-encoding mismatch	BUG-IDE-035: <code>tr = SHAKE256</code> of the 1952-B FPGA pk vs. the 3104-B Python packed pk	Hardware byte-diff against reference
Data-dependent crypto logic (norm-check off-by-N)	BUG-IDE-037: norm check read too few coefficients under BRAM latency; rare out-of-bound signature	8-hour adversarial soak; invisible to KAT vectors
Multi-block host truncation	On-chip keygen-wrap: <code>s2[1-5]</code> wrong; host returned on pk TLAST before on-chip recovery finished	Hardware byte-diff (last blocks wrong, rest exact)
Cross-process / driver transport	XDMA card-to-host (C2H) engine backlog (≈ 1041 B) surviving close→open; kernel-5.15 zero-padding partial C2H reads	Hardware A/B across hosts/kernels
Documentation over-claim	Stale “5–18× cycle gap” (unconfirmed comparators); stale “125 MHz” clock label in five deployed claims	Post-edit grep audit; review

Cache-dominated. 99.4% of processed tokens are prompt-cache reads and writes. The campaign’s ≈ 26 kLOC corresponds, at a conservative ~ 15 tokens per line, to under 2% of the generated tokens in this window alone (and much of that code predates the window). Scaling linearly by logged-activity share gives a campaign-order estimate of ~ 9 B tokens processed and ~ 44 M generated. The per-experiment token-estimate field is unreliable; the harness transcripts are ground truth.

I. Measured Productivity, Efficiency, and Accuracy

The numbers above map onto the three axes that matter to a team adopting the protocol of Section III-F (scope: H1):

- **Productivity**—one developer plus the assistant, ≈ 5 –6 months, $\approx 26,373$ LOC generated or modified, 232 logged experiments, a deployed accelerator + HSM.
- **Efficiency**—57.0% one-shot acceptance, mean 2.12 iterations, 21.6M generated tokens at 99.4% cache-mediated processing in the measured window (campaign-order ~ 44 M, an extrapolation from the 49% window).
- **Accuracy**—71.6% overall experiment success (77–85% documentation/research vs. 50–53% synthesis and bring-up), 6/6 operations byte-exact on silicon, and a 779,945-check adversarial soak with zero failures.

The productivity claim is the artifact’s existence and validation status (Section VIII, H1). On efficiency, the assistant *reads* two orders of magnitude more than it *writes* ($\approx 207:1$), so the compute cost is context, not generation. On accuracy, raw acceptance rates are the wrong lens; correctness comes from the validation spine, and the coupling gradient motivates the protocol’s human gating (Section III-F).

V. THE ARTIFACT AS EVIDENCE

The 232-experiment methodology produced a silicon artifact that scales past single modules to a complete deployed cryptosystem. Three bitstream labels recur: v87 (earlier deployed

build, resource baseline only), v88 (the frozen-corpus deployed baseline behind every Section IV hardware statistic), and v89 (the shipped release with FIPS-final wire formats, Section VI-E, Tables III–V). A full architectural exploration and side-channel analysis are out of scope (Section IX); Section VI adds the PCIe board, host-device protocol, and productized bitstream lineage.

A. Unified Architecture

As Figure 3 shows, the host communicates over AXI4-Lite (BAR0 control/status registers) and AXI4-Stream (host-to-card/card-to-host, H2C/C2H, bulk data) via Xilinx XDMA IP. Clock domains: 100 MHz core (`pqc_top_clk`), 250 MHz XDMA AXI (`axi_aclk`), 500 MHz PCIe transceiver (`userclk1`) (Section VI-A).

Shared NTT/INTT datapath. A single 5-stage pipelined butterfly (5-cycle latency, one butterfly per cycle after fill) iterates all 7 (ML-KEM) or 8 (ML-DSA) NTT layers; twiddle ROMs hold both moduli’s tables with the active modulus selected per transform, and the twiddle multiply is reduced by a two-cycle Montgomery [20] step. NTT-domain multiplication instead uses Barrett reduction [21]— $m = 5039$, $k = 24$ for ML-KEM ($q = 3329$) and $m = 8396807$, $k = 46$ for ML-DSA ($q = 8380417$)—keeping coefficients in the normal domain: a degree-one *base multiply* for ML-KEM’s incomplete NTT, a *pointwise multiply* for ML-DSA’s complete NTT.

Shared Keccak core. A single Keccak-f[1600] unit serves both control FSMs and the key-custody keywrap/deterministic random-bit generator (DRBG) through a registered two-master mux, whose one extra command-bus cycle, invisible to direct-wired unit testbenches, caused a wrong-tag failure in an intermediate custody-era build (v79c), caught only on hardware.

On-die entropy (HRNG). A hardware RNG (`qec_aggregator`) supplies all internal randomness. Four off-die Am-241 quantum-entropy chip sites (QEC [22]; one selected at runtime via `HRNG_CTRL[8:7]`), sampled

TABLE IV
SYSTEM THROUGHPUT AND LATENCY, SHIPPED V89, HOST-MEASURED OVER PCIe AT THE PEAK FAST-PATH LEVEL (INTEGRATED-SYSTEM NUMBERS, NOT CORE-CYCLE RECORDS). CUSTODY LATENCIES WERE MEASURED ON THE V88 BUILD.

Scheme	Operation	Throughput (ops/s)	Mean latency (ms)
ML-KEM-768	KeyGen	2667	0.375
	Encaps	1857	0.538
	Decaps	1377	0.726
ML-DSA-65	KeyGen	768	1.303
	Sign	480	2.083
	Verify	786	1.273
Custody latency	On-chip wrap	≈ 7 ms (provisioning, host-measured)	
	On-chip unwrap	≈ 0.6 ms (per sign-from-vault call)	
	Sign from vault	17–21 ms (unwrap + sign + blob transfer, reset/drain)	
	Decapsulate from vault	1–2 ms (incl. unwrap + decaps)	

the 0.726 ms measured mean. ML-DSA Signing is rejection-dependent at ≈ 0.4 – 0.6 M cycles; Table IV’s lower 2.083 ms (≈ 0.21 M cycles) reflects the benchmark’s fixed low-rejection message. A forward NTT takes 907 (ML-KEM) or 1035 (ML-DSA) cycles per polynomial.

D. BUG-IDE-037: The Methodology’s Key Catch

Detection and symptom. A later seed-driven randomized mixed-operation soak ($\approx 2,200$ -vector pool) caught a verification-failing signature for one message class (EXP-20260615-001); the preceding $\approx 84,000$ -check soak (EXP-20260614-001) passed clean, prompting episode 1 of Section IV-F; a constant SHA-256 hash across reruns proved the defect deterministic, not transient or thermal.

Root cause. In `dilithium_sign.v`, the rejection-sampling norm check reads coefficients from the main BRAM with 2-cycle latency, but the $\|z\|$, $\|r_0\|$, and $\|c \cdot t_0\|$ scan counts covered *exactly* the polynomial, so the last one or two coefficients arrived *after* the accept/reject decision. The failing vector (vec76, reject-loop iteration 5) hit this blind spot: its sole norm-violating coefficient, $r_0[\text{poly}[5]][\text{coeff}[0]]$, was last in scan order; a spec-correct signer rejects the candidate and first accepts at iteration 8.

Fix and validation. The fix extends each scan by the BRAM latency margin: z 1279 $\rightarrow$ 1281 (+2, wrapping), r_0 1535 $\rightarrow$ 1537 (+2, wrapping), and $c \cdot t_0$ 1535 $\rightarrow$ 1536 (+1, linear, plus a garbage-tail guard). On hardware the failing vector re-tested 20/20 byte-exact and the 7/7 KAT regression passed.

Detectability. Every affected signature *fails* verification (leakage caveats: H8, Section VIII). KAT vectors cannot trigger it: they never sample the message class that drives the rejection loop to the blind-spot iteration.

Reliability closure. An 8-hour soak (`c_exhaustive_8h_soak_hw`) ran 28,801.6 s, completing 779,945 checks with zero failures across ML-DSA provision (72,625), sign-from-vault $\rightarrow$ Verify on data-dependent messages (301,343), Verify tamper-reject (37,738), ML-KEM encaps $\rightarrow$ decaps-from-vault (295,673), and zeroize durability (72,562); ML-KEM provision (4) is thin coverage, a stated caveat. A separate C-library soak added 186,432 checks, also with zero failures.

E. Comparison with Prior Work

All comparator figures in Table V were verified against primary sources (IACR ePrint, TCHES/PeerJ/arXiv open-access versions).

Why A0’s latency is larger. A0 is roughly an order of magnitude slower per operation for three compounding reasons. (i) *Measurement scope: end-to-end system latency* over PCIe versus the prior rows’ accelerator-core latency. (ii) *Clock:* A0’s PQC core runs at 100 MHz on a Kintex-7 versus 270–375 MHz UltraScale+ parts. (iii) *Microarchitecture:* A0 *serializes* the NTT through a single time-shared 5-stage butterfly (Section V-A) to fit both schemes *plus* key custody in a nearly full XC7K160T, so even in clock-independent core cycles it spends ~ 6 – $12\times$ more than the parallel comparators (ML-KEM Decaps $\approx 67,550$ vs. ≈ 5.5 – 11 k). With AVX2 software below the PCIe round-trip floor, a PCIe accelerator cannot win on raw throughput *by construction*; A0’s value is *CPU offload and hardware key custody*, which no compared core provides.

VI. THE PQC-PCIe BOARD IMPLEMENTATION

The artifact of Section V is a deployed PCIe card with a complete host stack. Most of the hardware difficulty sits here: the 50% integration/bring-up bucket (Table I) and much of the 2026-05 dip to 41% (Section IV-G) trace to Section VI-C.

A. Board and PCIe Subsystem

The board (EYL “PQC-HSM K160,” model K160-PCIe, a working product designation) carries the XC7K160T-FFG676-2 behind a PCIe edge connector, a Micron MT25QL128 SPI configuration flash, and the Am-241 quantum-entropy subsystem of Section V-A. Deployed builds draw ≈ 5.4 – 5.6 W on-chip (≈ 1.5 – 1.65 W in the PQC core, the rest mostly PCIe/XDMA and transceivers).

Host attachment uses the AMD/Xilinx XDMA endpoint [28] at Gen2 5.0 GT/s $\times 8$ (≈ 3.0 GB/s modeled effective DMA), which exposes a 64-kB BAR0 window of 32-bit AXI4-Lite registers plus one H2C and one C2H AXI4-Stream channel (128-bit at 250 MHz), width-adapted to the wrapper’s 8-bit interface across three clock-domain crossings into the 100 MHz core domain. The XDMA-internal 500 MHz `userclk1`, chronically the tightest domain (the timing-closure row of Table II), closed between $+0.041$ and $+0.049$ ns.

TABLE V

SELECTED UNIFIED ML-KEM+ML-DSA FPGA ACCELERATORS, PER-OPERATION LATENCY IN μ S. LETTERED MARKS REFER TO THE NOTES BENEATH THE TABLE. ROW A0 (THIS WORK) IS *end-to-end system* LATENCY; THE FPGA COMPARATOR ROWS ARE *accelerator-core* LATENCY AT $\approx 2.7\text{--}3.75\times$ HIGHER CLOCKS, SO THE TWO ARE *not directly comparable*.

#	Work	Device	$F_{\max}$ (MHz)	LUT	DSP	BRAM	ML-KEM-768 KG/Enc/Dec	ML-DSA-65 KG/Sign/Vrf
<i>Unified Kyber + Dilithium accelerators (closest comparators)</i>								
A0	This work (v89)	Kintex-7 XC7K160T	100/500 ^a	88,478 ^b	39 ^b	171.5 ^b	375/538/726 ⁱ	1303/2083/1273 ⁱ
A1	KaLi [23]	ZCU102 ^c	270	23,277	4	24	23.2/29.11/41.82	87.5/179.9/96.8 ^d
A2	Dobias et al. [24]	ZCU102 ^c	375	17,138	4	12.5	9.4/10.9/14.8	41.7/88.5/44.6
A3	KiD [25]	ZU+/A7/Z7 ^e	294–342	2,893–5,909	4–8	4.5–5.5	<i>NTT multiplier unit only</i> ^f	
A4	Wang et al. [26]	PolarFire SoC ^g	—	<5% dev.	—	—	3–5 $\times$ SW speedup	3–5 $\times$ SW speedup
A5	Sapphire [27]	40 nm ASIC ^h	72	106 kGE	—	—	<i>configurable multi-scheme</i> ^h	

^a 100MHz core clock, 500MHz PCIe transceiver clock. ^b Device utilization: LUT 87.3%, DSP 6.5%, BRAM 52.8%. ^c ZCU102 is Zynq UltraScale+. ^d Best-case figures. ^e Zynq UltraScale+/Artix-7/Zynq-7. ^f NTT multiplier unit only, not a full KEM or signature core; conflict-free dual-port. ^g PolarFire SoC (RISC-V); resource figure is device fraction. ^h ASIC, not FPGA; area in kGE; configurable multi-scheme (Kyber/Dilithium-class, RISC-V + NTT + SHA-3). ⁱ End-to-end *system* latency (host + PCIe Gen2 $\times$ 8 + DMA), v89 measured at the peak fast path; the default correctness-first path adds ≈ 1 ms of inter-operation reset and drain. Includes on-chip key custody (DNA-rooted KEK, wrap, sign/decaps-from-vault). Prior-work rows are *accelerator-core* latency and are not directly comparable, since clock rates, host interfaces, and custody overhead all differ. Core-cycle anchors and the Sign low-rejection caveat: Section V-C.

Reset is asymmetric: the BAR0 soft-reset (0×08 , added in v29) resets all PQC sub-FSMs via a self-clearing 16-cycle pulse ($\approx 50\text{--}100\mu$ s host-observed, including the re-arm poll), but the XDMA DMA engines and H2C CDC FIFO reset only on PCIe Function-Level Reset (FLR), and everything on the FLR-only side eventually produced a bug. The physical workflow (Vivado Lab 2023.2 flashing, a cold power-cycle after every flash, an XDMA kernel module rebuilt against the running kernel) was learned failure by failure and encoded into the persistent memory stack (Section III-B).

B. Host–Device Protocol

Control uses a compact bank of BAR0 registers (CTRL, STATUS, SOFT_RESET, the HRNG_* entropy and HSM_* custody banks, and Device-DNA readback; full map in the released manual). Dispatch is deliberately minimal: the host writes a 3-bit opcode to CTRL, streams input over H2C, and reads the fixed-length result over C2H, with no framing headers or length fields (t_{last} delimits); the C2H read returning its expected byte count *is* the completion signal, since STATUS.done is a one-cycle pulse no PCIe-latency poll can catch (interrupts are unused), and STATUS.busy is busy-pollled only as the re-arm gate. Wire sizes (v89 FIPS-final, H2C $\rightarrow$ C2H, bytes): ML-KEM KeyGen 32 $\rightarrow$ 1184, Encaps 1216 $\rightarrow$ 1120, Decaps 3488 $\rightarrow$ 32; ML-DSA KeyGen 32 $\rightarrow$ 1952, Sign 4064 $\rightarrow$ 3309, Verify 5293 $\rightarrow$ 0. In internal-seed mode (HRNG_CTRL[9]) KeyGen seed bytes drop to zero and custody provisioning returns a wrapped blob (nonce + tag + ciphertext: 4096 B for ML-DSA-65, 2464 B for ML-KEM-768). The direct Sign/Decaps formats carry the plaintext secret key over H2C (the compute/migration path); custody sign/decaps-from-vault instead re-presents the host-held, on-die-wrapped blob over BAR0 vault writes on each call and unwraps it on-die (the blob-transfer term of Table IV). The one exception to C2H result readback, ML-DSA Verify, reports its result in STATUS[2] dil_valid, a late discovery.

The protocol also carries five *operational disciplines*, none in any component datasheet—each a distilled, logged silicon failure, protocol step P5 made visible:

- D1. Soft-reset + DVF_STATE idle poll before every Verify** (DVF_STATE taps the Verify FSM’s live state over BAR0): else the second back-to-back Verify returns invalid *deterministically* (200/200; BUG-IDE-036, Section IV-F).
- D2. Inter-operation settle:** consecutive ML-DSA operations need ≥ 1 ms separation. The fast path uses a busy-poll re-arm gate plus an unconditional 0.5 ms residue drain.
- D3. Warm-up burn-op at open:** a ≈ 1 kB tail in the XDMA C2H engine survives close() $\rightarrow$ open() and soft-reset, shifting the next process’s first result by one byte; one throwaway KeyGen flushes it.
- D4. FLR as transport recovery:** a wedged H2C path, unreachable by any drain or soft-reset, recovers by scripted PCIe FLR in ≈ 6 s, no power-cycle.
- D5. Single-opener serialization:** the XDMA device admits one opener (EBUSY). Multi-client use goes through the pqcd daemon (Section VI-D).

C. The Bring-Up Record: Where the 50% Bucket Was Earned

The PCIe bring-up compressed into four weeks (bitstreams v11–v58b): first Gen2 $\times$ 8 builds on 2026-05-04 (v11), scripted flash on 05-17 (v13), first end-to-end PCIe operation (ML-KEM KeyGen, v18) and first byte-exact result (v22) on 05-21 after fixing X-initialized \$readmemh ROMs and the Vivado multi-write BRAM dissolution, and all six operations FSM-complete on 05-22 (v29). Byte-exactness climbed 4/6, 5/6, 6/6 across 05-26/-27/-28 (the last a host-side fix, no reflash) and reached 6/6 at full back-to-back rate on 05-29 (mitigated host-side; D1).

Three failure families dominate the record (Table II):

(a) **Stream-handshake races in RTL.** Combinational request lines into register-derived readiness, one-cycle pulses against back-pressure, target-switch first-byte duplication, and

compute-gap over-fetch all passed iverilog and failed on silicon; each fix was a registered request, a sticky flag, or a one-deep skid buffer.

(b) Host-vs-RTL misdiagnosis. The costliest family, and the observability problem in its purest form: host, driver, DMA, and RTL produce identical boundary symptoms, so byte-exactness locates a failure without attributing it. Three “RTL bugs” were host code: an abandoned reader thread silently consuming device bytes (BUG-IDE-031); a C2H drain that silently no-ops off the main thread (BUG-IDE-033); and BUG-IDE-034, the host waiting for Verify result bytes the RTL never emits on C2H—masking two RTL repair attempts (v49, v51) until the register-level read was found on 05-27, its resolution exposing a further host-side `tr`-encoding mismatch (BUG-IDE-035). The inverse occurred once: the back-to-back defect (BUG-IDE-036, D1) presented as a driver flake but is RTL-behavioral. Attribution came only from differential A/B on live hardware (direct vs. threaded, C vs. Python, before vs. after FLR), distilled into standing rules: a failure that follows the host-side variable indicts the host, and no RTL hunt opens before a clean re-run. An X-clean full-module simulation, cheap via the golden-reference spine (Section III-D), exonerated the single-operation Verify datapath as correct since v51.

(c) Transport-layer defects. XDMA C2H short-completions, AXI-Stream padding misreads, and a ≈ 1 -in-20 mixed-operation H2C wedge (`errno 512`, FLR-only) round out the family; the wedge was self-healed by the soak harness for weeks until root-caused (BUG-IDE-021). Its fix took a 500-iteration A/B from 24 wedges to one, then zero across a 1,500-iteration confirmation, 12,825/12,825 byte-exact.

The methodology reading: almost nothing here is cryptographic—it is boundary engineering, exactly where Table I shows coin-flip reliability (Section IV-D).

D. Host Software Stack

Four host surfaces ship with the board, all implementing the protocol and disciplines above. A Python library (`pqc_pcie_host.py`) covers all six operations plus custody, D3 warm-up by default. A dependency-free C99 library (`libpqchsm`) is protocol-exact against it and is the Table IV bench client. The `pqcd` daemon serializes clients over the single-opener device (D5) and feeds a reference OpenSSL-3 provider that signed a TLS-1.3 CertificateVerify through hardware custody. The demo/validation suites add the `check_all_six` byte-exact gate and the self-healing soak harness of Section V-D. The redundancy is deliberate: C/Python divergences repeatedly localized transport bugs neither stack could attribute alone (Section IV-E).

E. Productization and Bitstream Lineage Beyond the Snapshot

Release v1.0 (first cut 2026-06-21 on bitstream v87) packaged the bitstream, host libraries, daemon, suites, and manual. **Release v1.1** (2026-07-08) ships bitstream v89, converting both schemes from round-3-derived encodings to the final published standards: FIPS-203/204 key-generation

seed derivations, 4-bit SimpleBitPack w_1 encoding, a 48-byte challenge, a signature grown from 3293 to 3309 bytes (the only wire-visible length change), and FIPS-204 domain-separated message hashing. With v89, device signatures are byte-identical to the FIPS-204 wire format, validated against the `dilithium-py` oracle, so conforming external verifiers (e.g., `liboqs` [29], `OpenSSL`) accept them with no host-side converter, and the device verifies external signatures directly. Release v1.1 was re-validated on the board: full suite set and adversarial soak, zero failures on v89.

Every corpus-derived Section IV statistic is quoted at the frozen 2026-06-26 snapshot (v88); the one flagged exception is Section IV-H’s measurement window (six experiments beyond the freeze). Release v1.1 postdates the snapshot; built under the same logged methodology, it stands as lineage, not analyzed corpus (scoping: H3). Release v1.0 and its v88 update (2026-06-24) preceded the freeze by days, and their logs are counted inside the frozen corpus (the release/infra/PM bucket of Table I).

VII. RELATED WORK

A. LLMs for RTL Generation and Hardware Benchmarking

Thakur et al. [7] introduced RTL-generation benchmarks, VerilogEval [8] formalized `pass@k` evaluation, and RTLCoder [9] contributed an open fine-tuned model and training set; ChipNeMo [38] domain-adapts models, and Chip-Chat [39] carried an LLM-co-designed microcontroller through a SkyWater 130 nm shuttle tapeout. The artifacts are snippets, modules, or that shuttle core validated at simulation, lint, bench demo, or expert rating; none reaches hardened, in-service silicon with a longitudinal outcome record.

B. Agentic and Feedback-Driven LLM Hardware Flows

A second line closes a tool-in-the-loop cycle: AutoChip [10] iterates on testbench feedback, RTLFixer [11] repairs syntax, MEIC [17] and VerilogCoder [12] add functional debugging and planning, AssertLLM [18] generates assertions, and ChatEDA [13] orchestrates the EDA toolchain on benchmark designs (survey: [40]). None reaches validated operation on deployed silicon. QiMeng [41] spans RISC-V CPU spec-to-silicon but stops at bring-up of a demonstrator taped out by its pre-LLM pipeline.

C. LLMs for Hardware Security

Ahmad et al. [19] use LLMs to fix security bugs in existing RTL rather than *synthesize* cryptographic hardware (survey: [42]); to our knowledge, no published work carries LLM-driven design with a cryptographic end-goal to validated operation on real silicon.

D. Closest Prior Art: LLMs for PQC Hardware

Two 2026 works are closest: LLM4PQC [14] and Liao et al. [15], the latter reporting up to $2.6\times$ kernel speedup over an HLS baseline. Both stop short of validated operation, whereas ours continues through byte-exact silicon and adversarial soak; Table VI details the five-axis delta.

TABLE VI
CLOSEST PRIOR WORKS APPLYING LLMs TO PQC HARDWARE; LAST COLUMN: THE SECTION SUBSTANTIATING EACH AXIS.

Axis	Liao et al. [15] (ISQED’26)	LLM4PQC [14] (DATE’26)	This work	Sec.
Scope	Compute kernels (FALCON)	PQC cores via HLS refactor- ing	Complete cryptosystem with host protocol, HSM, and bring-up	V, VI
Deployment	FPGA implementation, ker- nel level	Synthesizable RTL	Real Kintex-7 silicon over PCIe Gen2 $\times 8$	VI
Key custody	—	—	On-chip custody for both schemes (HRNG-seeded)	V
Interaction model	Human-in-the-loop prompt it- eration	Feedback-driven, agentic	Agentic CLI with file/shell tools, subagents	III
Dataset	—	—	232 structured experiment logs	IV

TABLE VII
COMMERCIAL PQC SILICON (MID-2026). FW = PQC AS FIRMWARE ON A
SECURE PROCESSOR (NO DEDICATED DATAPATH).

Product	Class	PQC in silicon	Status
ST ST54M [30]	mobile SE	ML-KEM, ML-DSA	sampling 2026
Samsung S3SSE2A [31]	embedded SE	ML-DSA-65 only	sampling 2026
SEALSQ QS7001 [32]	IoT SE	ML-KEM, ML-DSA	shipping 2025
Microchip MEC175xB [33]	embedded ctrl. (RoT)	ML-KEM, ML-DSA, LMS	sampling 2025
Lattice MachXO5-NX TDQ [34]	secure-control FPGA	ML-KEM; LMS/XMSS bitstream auth	shipping 2025
Caliptra / Adams Bridge [35]	RoT IP (data- center SoC)	ML-DSA-87 + ML- KEM-1024, unified	RTL 2025 (IP-only)
IBM Crypto Express 8S [36]	PCIe HSM	none (Kyber / Dilithium in CCA FW)	shipping 2022
Entrust nShield 5 [37]	PCIe / net. HSM	ML-KEM / ML- DSA / SLH-DSA; internal-FPGA accel. of undisclosed per- algorithm scope	FW 13.8, 2025
This work	PCIe HSM ac- cel.	ML-KEM-768+ML- DSA-65 full datapaths, on-die custody	deployed (v89)

E. ML-KEM/ML-DSA Hardware Accelerators

FPGA acceleration of CRYSTALS-Kyber (ML-KEM, FIPS 203 [1], [2]) spans a Pareto front from Xing and Li’s compact 2-DSP/3-BRAM Artix-7 design [43] to the pipelined HPKA [44]; high-speed ML-DSA designs use parallel butterflies and multiple Keccak cores (2×2 NTT [45]); area-leaner designs cover all round-3 parameter sets [46], with compact variants at the far end of the area axis [47], [48].

Most directly comparable are unified Kyber+Dilithium designs: KaLi [23] (first, ZCU102, 270 MHz), Dobias et al. [24] (most resource-efficient to date, 17,138 LUT at 375 MHz), KiD [25], Wang et al. [26], Carril et al. [49] (HLS batch acceleration on a PCIe datacenter Alveo card), Beckwith et al. [50], and Truong et al. [51] (both FIPS-final standards, all phases, Zynq UltraScale+). Ours is (1) the only one, to our knowledge, on a mid-range Kintex-7; (2) integrated with on-chip key custody *and* HRNG-seeded on-die key generation for both schemes (prior unified designs receive seeds and keys over the host bus); and (3) measured end-to-end over PCIe including host-driver and custody overhead, not at the accelerator boundary. PUF-Dilithium [52] also targets on-chip key custody but stops at PUF-based seed protection of ARM software. A complementary line masks datapaths (among

unified designs, Beckwith et al. [50] report SCA protection, and Dobias et al. [53] a first-order-masked, TVLA-evaluated datapath); this work deliberately makes no side-channel claim (Section VIII, H6).

Commercial PQC silicon. Table VII shows dedicated PQC silicon arriving in 2025–mid-2026. Caliptra’s open-source Adams Bridge accelerator unifies ML-DSA-87 and ML-KEM-1024 as licensable datacenter-SoC IP [35], the same consolidation this design applies. At HSM scale, most shipping products run PQC as firmware on general-purpose secure processors [36]; the closest analogue is Entrust’s nShield 5, a classical HSM field-upgraded with internal-FPGA PQC acceleration [37]. To our knowledge, no commercial product yet documents full hardware datapaths for both an ML-KEM and an ML-DSA parameter set with on-die key custody at host-attached (PCIe) HSM scale; that is this artifact’s slot.

VIII. THREATS TO VALIDITY AND LIMITATIONS

We state nine honesty constraints—not hedges but integral to the scientific claim.

- (H1) *n=1 case study / self-reported logs.* No human-only baseline was run: the productivity and cost figures of Section IV-I are absolute, not comparative. Labels, lesson coding, and the per-log edit/iteration fields are self-recorded, incompletely covered (179 of 232; bounds in Section IV-D), and not independently adjudicated; a different segmentation would shift the percentages, not the coupling-gradient ordering. Acceptance stringency also differs by bucket (operator judgment at low coupling, byte-exact at high; Section IV-B), so part of the gradient reflects adjudication. Corpus completeness rests on contemporaneous logging (Section III-C); abandoned work scores as failure (Section IV-A).
- (H2) *Single LLM family / single operator.* All sessions used Claude models and one developer; within this tightly coupled dyad, contributions are not separable, and generalizability to other LLMs, toolchains, operators, or task mixes is unknown—the bucket populations of Table I reflect this project’s phase structure.
- (H3) *Trustworthiness from validation, not authorship.* Each RTL module is trusted because it passed byte-exact golden-reference comparison and, on the deployed v88 baseline, the 779,945-check adversarial soak; the v89 release re-passed the full ladder, soak included (Section VI-E).

- (H4) Device binding, not at-rest key secrecy.** Given 7-series bitstream-encryption weaknesses [54], the bitstream-embedded *KEK_SALT* cannot be assumed secret against a physical adversary; a hardened key store is future work (Section IX).
- (H5) Integrated-system measurements, not a throughput record.** Reported ops/s are measured at the peak fast-path level (the default correctness-first path adds ≈ 1 ms of inter-operation reset and drain), and the ML-DSA Sign figure reflects the benchmark’s fixed low-rejection message (Section V-C).
- (H6) No side-channel resistance claims.** No masking is applied to the NTT, Keccak, or sampling datapaths (future work: Section IX).
- (H7) Not FIPS 140-3 validated.** The module is FIPS 203/204 byte-exact against the reference implementations (Section VI-E) but has undergone neither Cryptographic Algorithm Validation Program (CAVP) testing nor FIPS 140-3 validation; likewise, while the QEC source’s min-entropy is characterized per SP 800-90B methodology [22], the deployed HRNG chain has no formal entropy-source validation—statistical batteries validate the conditioner output only.
- (H8) BUG-IDE-037 is fail-detectable, not a forgery vulnerability.** The defect produces signatures that fail verification (fixed in v84); the rare pre-fix emissions are treated as a potential leakage surface, with no claim of security-neutrality.
- (H9) Single-operation-at-a-time / XDMA single-opener.** ML-KEM and ML-DSA share the NTT and Keccak cores, so one operation executes at a time; the XDMA device is single-opener (D5).

IX. CONCLUSION

The standard acceptance test for lattice-signature hardware cannot detect an entire class of defects: ML-DSA correctness is a property of control paths while known-answer tests sample values; BUG-IDE-037 passed a full KAT regression on deployed silicon. We resolved that blindness by replacing the gate rather than the author: acceptance decided by a byte-exact golden-reference oracle plus randomized adversarial soak—301,343 data-dependent signings among 779,945 checks with zero escapes on the deployed v88 baseline. Nothing in that construction depends on who wrote the RTL, which made the AI question answerable. Across 232 logged experiments an agentic LLM assistant drove RTL through hardening at 71.6% success, following a *hardware-coupling gradient* (77–85% documentation/research vs. 50–53% synthesis and bring-up). The mechanism is observability, not difficulty: reasoning over readable artifacts cannot recover a corrective signal that exists only on the physical side of the boundary. The slope of this $n=1$ study is a hypothesis with a mechanism attached, not a calibrated constant. Not in question is the artifact: a unified ML-KEM-768 and ML-DSA-65 accelerator with on-chip key custody on one mid-range XC7K160T at 98.5% slice occupancy and 49 ps of margin at 500 MHz, byte-exact across all six FIPS operations (shipped v89, re-soaked, zero failures).

Future directions. (i) A controlled multi-operator, multi-model replication with baselines and independent outcome adjudication beyond this $n=1$ case; (ii) genuine at-rest key-encryption-key secrecy via an eFUSE-protected UltraScale+ (XCZU7EV) port, already first-silicon byte-exact across all six operations under the same methodology; (iii) side-channel hardening by threshold implementation, where LLM-assisted masked-gadget generation and RTL-level leakage repair [55] could lower its cost; (iv) higher-throughput multi-butterfly NTTs on larger devices, in deployments where the core rather than the PCIe round trip bounds throughput; (v) the deferred custody-protocol specification and analysis, plus FIPS 140-3 and CAVP validation for regulated deployments; and (vi) context condensation—distilled digests of the slowly changing corpus to cut the context cost center directly (Section IV-I), in measured tension with the stale-prose lesson of Section IV-F. All inherit the same discipline: state the rung of validation that earns the claim, and claim nothing above it.

AUTHOR CONTRIBUTIONS AND ACKNOWLEDGMENTS

J. Park conducted the 232-experiment campaign as the single operator and wrote the manuscript; E. Kim, W. Kim, and S. Cho (EYL Inc.) provided and supported the PQC-PCIe K160 board platform; B. Cha critically reviewed and revised the manuscript. Anthropic Claude models served as the LLM assistant under the human-gated methodology of Section III.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” U.S. Department of Commerce, NIST, Tech. Rep. FIPS 203, Aug. 2024.
- [2] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM,” in *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018, pp. 353–367.
- [3] National Institute of Standards and Technology, “Module-lattice-based digital signature standard,” U.S. Department of Commerce, NIST, Tech. Rep. FIPS 204, Aug. 2024.
- [4] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Dilithium: A lattice-based digital signature scheme,” *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2018, no. 1, pp. 238–268, 2018.
- [5] National Security Agency, “Announcing the Commercial National Security Algorithm Suite 2.0,” Cybersecurity Advisory, 2022, updated 2025.
- [6] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, “Transition to post-quantum cryptography standards,” NIST, Tech. Rep. NIST IR 8547 (initial public draft), 2024.
- [7] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, “Benchmarking large language models for automated Verilog RTL code generation,” in *Design, Automation & Test in Europe Conf. (DATE)*, 2023.
- [8] M. Liu, N. Pinckney, B. Khailany, and H. Ren, “VerilogEval: Evaluating large language models for Verilog code generation,” in *IEEE/ACM ICCAD*, 2023.
- [9] S. Liu, W. Fang, Y. Lu, J. Wang, Q. Zhang, H. Zhang, and Z. Xie, “RTLCode: Fully open-source and efficient LLM-assisted RTL code generation technique,” *IEEE TCAD*, vol. 44, no. 4, pp. 1448–1461, 2025.
- [10] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, “AutoChip: Automating HDL generation using LLM feedback,” arXiv:2311.04887, 2023.
- [11] Y.-D. Tsai, M. Liu, and H. Ren, “RTLFixer: Automatically fixing RTL syntax errors with large language models,” in *ACM/IEEE DAC*, 2024.
- [12] C.-T. Ho, H. Ren, and B. Khailany, “VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool,” arXiv:2408.08927, 2024.

- [13] H. Wu, Z. He, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu, "ChatEDA: A large language model powered autonomous agent for EDA," *IEEE TCAD*, vol. 43, no. 10, pp. 3184–3197, 2024.
- [14] B. Perera, Z. Wang, W. Xiao, M. Nabeel, O. Sinanoglu, J. Knechtel, and R. Karri, "LLM4PQC: Accurate and efficient synthesis of PQC cores by feedback-driven LLMs," in *Design, Automation & Test in Europe Conf. (DATE)*, 2026.
- [15] Y. Liao, T. Adegbiya, and R. Lysecky, "Accelerating post-quantum cryptography via LLM-driven hardware-software co-design," in *2026 27th International Symposium on Quality Electronic Design (ISQED)*, 2026.
- [16] National Institute of Standards and Technology, "SHA-3 standard: Permutation-based hash and extendable-output functions," U.S. Department of Commerce, NIST, Tech. Rep. FIPS 202, 2015.
- [17] K. Xu, J. Sun, Y. Hu, X. Fang, W. Shan, X. Wang, and Z. Jiang, "MEIC: Re-thinking RTL debug automation using LLMs," arXiv:2405.06840, 2024.
- [18] W. Fang, M. Li, M. Li, Z. Yan, S. Liu, H. Zhang, and Z. Xie, "WAssertLLM: Generating hardware verification assertions from design specifications via multi-LLMs," in *ASP-DAC*, 2025.
- [19] B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "On hardware security bug code fixes by prompting large language models," *IEEE TIFS*, vol. 19, pp. 4043–4057, 2024.
- [20] P. L. Montgomery, "Modular multiplication without trial division," *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [21] P. Barrett, "Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor," in *CRYPTO 1986*, ser. LNCS, vol. 263. Springer, 1987, pp. 311–323.
- [22] J. Park, S. Cho, T. Lim, and M. Tehranipoor, "QEC: A quantum entropy chip and its applications," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 6, pp. 1471–1484, 2020.
- [23] A. Aikata, A. C. Mert, M. Imran, S. Pagliarini, and S. S. Roy, "KaLi: A crystal for post-quantum security using Kyber and Dilithium," *IEEE Trans. Circuits Syst. I*, vol. 70, no. 2, pp. 747–758, 2023.
- [24] P. Dobias, L. Malina, and J. Hajny, "Efficient unified architecture for post-quantum cryptography: Combining Dilithium and Kyber," *PeerJ Computer Science*, vol. 11, p. e2746, 2025.
- [25] S. Mandal and D. Basu Roy, "KiD: A hardware design framework targeting unified NTT multiplication for CRYSTALS-Kyber and CRYSTALS-Dilithium on FPGA," arXiv:2311.04581, 2023.
- [26] T. Wang, C. Zhang, X. Zhang, D. Gu, and P. Cao, "Optimized hardware-software co-design for Kyber and Dilithium on RISC-V SoC FPGA," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2024, no. 3, pp. 99–135, 2024.
- [27] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, "Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2019, no. 4, pp. 17–61, 2019.
- [28] AMD/Xilinx, "DMA/Bridge Subsystem for PCI Express (XDMA) Product Guide, PG195 (v4.1)," <https://docs.amd.com/r/en-US/pg195-pcie-dma>, 2023.
- [29] Open Quantum Safe Project, "liboqs: C library for quantum-safe cryptography," <https://openquantumsafe.org>, 2026.
- [30] STMicroelectronics, "STMicroelectronics unveils world's first ST54M secure mobile chip with post-quantum cryptography for next-generation connected services," <https://newsroom.st.com/media-center/press-item.html/p4784.html>, 2026.
- [31] Samsung Semiconductor, "S3SSE2A: Hardware PQC locks in security for the quantum era," <https://semiconductor.samsung.com/news-events/tech-blog/s3sse2a-hardware-pqc-locks-in-security-for-the-quantum-era/>, 2025.
- [32] SEALSQ, "QS7001 post-quantum secure element — summary datasheet," <https://alcom.be/uploads/SealSQ-QS7001-Datasheet.pdf>, 2024.
- [33] Microchip Technology, "MEC175xB embedded controller family: Quantum-resistant security for the next generation of computing (brochure DS00005905A)," <https://ww1.microchip.com/downloads/aemDocuments/documents/SCBU/ProductDocuments/Brochures/r4-00005905A.pdf>, 2025.
- [34] Lattice Semiconductor, "Lattice launches industry-first PQC-ready FPGA family: MachXO5-NX TDQ," <https://latticesemi.gcs-web.com/news-releases/news-release-details/lattice-launches-industry-first-pqc-ready-fpga-family-machxo5-nx>, 2025.
- [35] M. Bisheh-Niasar, E. Karabulut, K. Upadhyayula, M. Norris, and B. Pillilli, "Adams Bridge Accelerator: Bridging the post-quantum transition," *Cryptology ePrint Archive*, Paper 2026/256, 2026.
- [36] IBM, "IBM 4770 PCIe cryptographic coprocessor (Crypto Express 8S) data sheet," https://public.dhe.ibm.com/security/cryptocards/pciecc5/doCs/4770_Data_Sheet.pdf, 2022.
- [37] Entrust, "Provide NIST-approved post-quantum algorithms in future-ready HSMs," <https://www.entrust.com/blog/2025/05/provide-nist-approved-post-quantum-algorithms-in-future-ready-hsms>, 2025.
- [38] M. Liu, T.-D. Ène, R. Kirby *et al.*, "ChipNeMo: Domain-adapted LLMs for chip design," arXiv:2311.00176, 2023.
- [39] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-Chat: Challenges and opportunities in conversational hardware design," in *2023 ACM/IEEE MLCAD*, 2023, pp. 1–6.
- [40] M. Abdollahi, S. F. Yeganli, M. A. Baharloo, and A. Baniasadi, "Hardware design and verification with large language models: A scoping review, challenges, and open issues," *Electronics*, vol. 14, no. 1, p. 120, 2025.
- [41] R. Zhang *et al.*, "QiMeng: Fully automated hardware and software design for processor chip," arXiv:2506.05007, 2025.
- [42] M. Akyash and H. M. Kamali, "Evolutionary large language models for hardware security: A comparative survey," in *GLSVLSI*, 2024.
- [43] Y. Xing and S. Li, "A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2021, no. 2, pp. 328–356, 2021.
- [44] Z. Ni, A. Khalid, D.-e.-S. Kundi, M. O'Neill, and W. Liu, "HPKA: A high-performance CRYSTALS-Kyber accelerator exploring efficient pipelining," *IEEE Trans. Computers*, 2023.
- [45] L. Beckwith, D. T. Nguyen, and K. Gaj, "High-performance hardware implementation of CRYSTALS-Dilithium," in *Int. Conf. Field-Programmable Technology (FPT)*, 2021, pp. 1–10.
- [46] G. Land, P. Sasdrich, and T. Güneysu, "A hard crystal—implementing Dilithium on reconfigurable hardware," in *Smart Card Research and Advanced Applications (CARDIS 2021)*, ser. LNCS, vol. 13173. Springer, 2022, pp. 210–230.
- [47] C. Zhao, N. Zhang, H. Wang, B. Yang, W. Zhu, Z. Li, M. Zhu, S. Yin, S. Wei, and L. Liu, "A compact and high-performance hardware architecture for CRYSTALS-Dilithium," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2022, no. 1, pp. 270–295, 2022.
- [48] N. Gupta, A. Jati, A. Chattopadhyay, and G. Jha, "Lightweight hardware accelerator for post-quantum digital signature CRYSTALS-Dilithium," *IEEE Trans. Circuits Syst. I*, vol. 70, no. 8, pp. 3234–3243, 2023.
- [49] X. Carril, C. Kardaris, J. Ribes-González, O. Farràs, C. Hernandez, V. Kostalabros, J. U. González-Jiménez, and M. Moretó, "Hardware acceleration for high-volume operations of CRYSTALS-Kyber and CRYSTALS-Dilithium," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 17, no. 3, pp. 41:1–41:26, 2024.
- [50] L. Beckwith, A. Abdulgadir, and R. Azarderakhsh, "A flexible shared hardware accelerator for NIST-recommended algorithms CRYSTALS-Kyber and CRYSTALS-Dilithium with SCA protection," in *Topics in Cryptology – CT-RSA 2023*, ser. LNCS, vol. 13871. Springer, 2023.
- [51] Q. D. Truong, Y. Jang, and H. Lee, "High-performance unified hardware architecture for ML-DSA and ML-KEM PQC standards," *IEEE Access*, vol. 13, 2025.
- [52] S. Aghapour, K. Ahmadi, M. Anastasova, R. Azarderakhsh, and M. Mozaffari Kermani, "PUF-Dilithium: Design of a PUF-based Dilithium architecture benchmarked on ARM processors," *ACM Trans. Embedded Comput. Syst.*, 2025.
- [53] P. Dobias, A. Aikata, A. Rezaeezade, L. Batina, L. Malina, and T. Gerlich, "Two birds, one mask: Side-channel-resistant unified hardware for ML-DSA and ML-KEM," *PeerJ Computer Science*, vol. 12, p. e4003, 2026.
- [54] M. Ender, A. Moradi, and C. Paar, "The unpatchable silicon: A full break of the bitstream encryption of Xilinx 7-series FPGAs," in *USENIX Security Symp.*, 2020, pp. 1803–1819.
- [55] A. Srivastava, S. Das, N. Choudhury, R. Psiakis, P. H. Silva, D. Pal, and K. Basu, "SCAR: Power side-channel analysis at RTL-level," arXiv:2310.06257, 2023.